

IBM z/OS Support for Infrastructure as Code

Ansible does not manage z/OS instances directly. It invokes system management facilities that are already present on the instance.

You *could* sign on to the z/OS instance and use those facilities directly. With Ansible, you're doing the same thing over a secure shell connection.

Using Ansible, you can manage multiple z/OS nodes, as well as specific z/OS subsystems on those nodes. Also, you can use the same tool stack to manage any or all Unix, Linux, Windows, and z/OS systems in the environment.

Let's briefly go over the system management support that ships with z/OS and where you can find the documentation.





Z/OSMF: Z/OS MANAGEMENT FACILITY

- Browser-based UI for managing z/OS
- REST APIs for managing z/OS
- Workflows for guided, repeatable configuration tasks
- SMP/E (Software Management) for installs and maintenance
- System status monitoring (LPARs, jobs, data sets, resources)

Z/OSMF WORKFLOWS

- Configuration as code (XML)
- Security configuration
- LPAR startup
- System customization
- Subsystem configuration (CICS, DB2, MQ)

Z/OS PROVISIONING

- Provision complete z/OS environments automatically
- Template-based (software + system definitions)
- LPAR provisioning
- z/OS instance provisioning
- Subsystem provisioning (CICS, DB2, MQ)
- REST APIs to enable CI/CD-style automation

SA Z/OS (SYSTEM AUTOMATION FOR Z/OS)

- Automatically start, stop, and restart subsystems
- Detect failures and take corrective action
- Manage dependencies between components
- Based on automation policies rather than scripts
- Can self-correct common error conditions

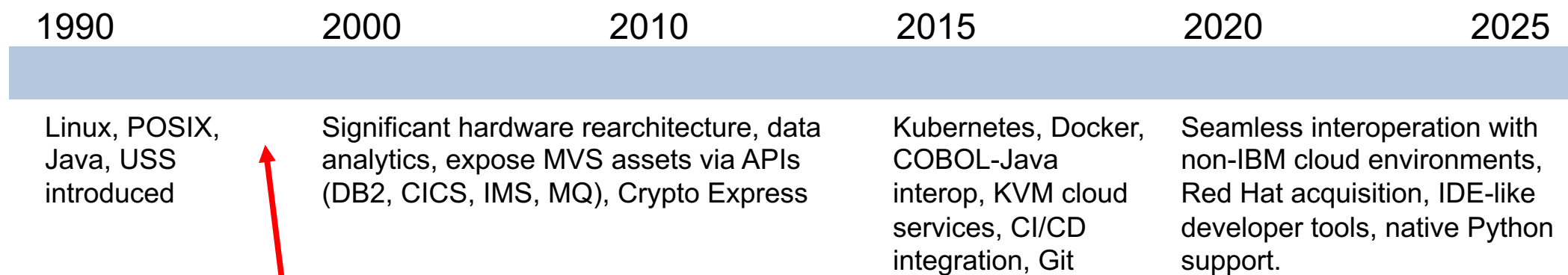
ADDITIONAL FEATURES AND FACILITIES

- GDPS (Geographically-Dispersed Parallel Sysplex)
- Hardware-level automation
- Dynamic workload management (Workload Manager)
- Based on automation policies rather than scripts
- Monitoring and event automation

WHAT DOES ANSIBLE ADD?

- Abstraction layer and interface younger professionals may find more comfortable than direct interaction with z/OS
- Consistent infrastructure management tooling across mainframe and non-mainframe assets in the enterprise
- Ansible invokes functionality of existing z/OS-based system management features rather than providing unique functionality
- Caveat: A clear understanding of the underlying z/OS automation features is necessary

ANSIBLE IN THE CONTEXT OF MODERNIZATION



Infoworld predicted the last mainframe would be unplugged on March 15, 1996

- IBM's Mainframe Modernization program began around 2000 with enhancements to the hardware and z/OS support for dynamic operations management and operational data collection and analysis.
- In the mid-2010s, the program began to address the growing problem that younger professionals were not comfortable working in the relatively primitive environment of TSO and ISPF. IDE plug-ins, mainly for Eclipse, began to appear.
- With the introduction of Zowe by Broadcom, Inc., the preferred developer front-end for z/OS became a bundle of VSCode extensions from IBM which includes Zowe, giving VSCode a level of z/OS awareness and giving developers an IDE-like experience. This requires z/OS components to be installed, as well; it isn't "download and run."
- Tools for system administrators and infrastructure engineers use either HTTP (browser UIs), RESTful APIs, z/OS Connect, or SSH to front-end z/OS system management facilities. The philosophy is the same as that of Zowe: to provide user interfaces to z/OS that resemble well-known interfaces for other systems.

MANAGED NODES ON A Z/OS SYSTEM ARE LPARS

all:

children:

zos:

hosts:

zoslp01:

ansible_host: zoslp01.company.com

sysplex: PLEXA

zoslp02:

ansible_host: zoslp02.company.com

sysplex: PLEXA

zoslp03:

ansible_host: zoslp03.company.com

sysplex: PLEXB

zoslp04:

ansible_host: zoslp04.company.com

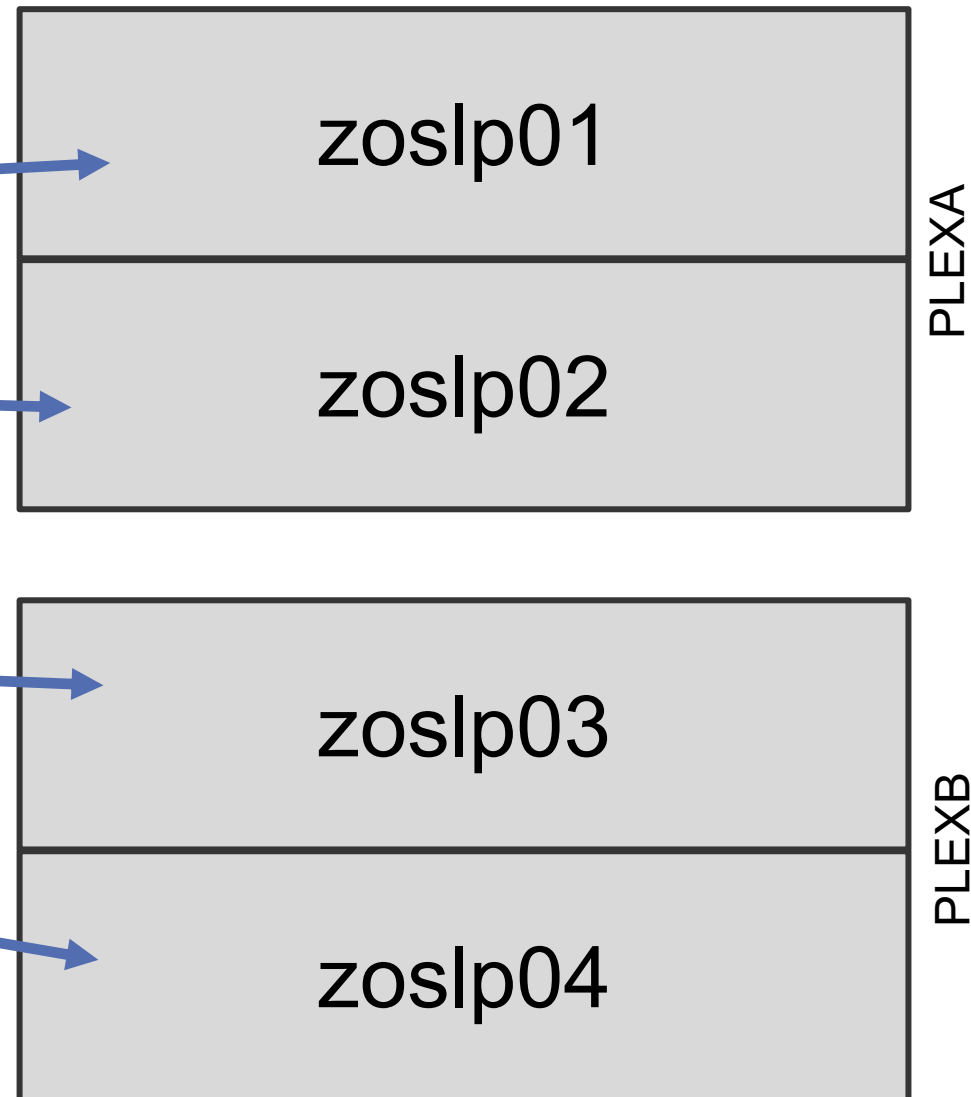
sysplex: PLEXB

vars:

ansible_user: ibmuser

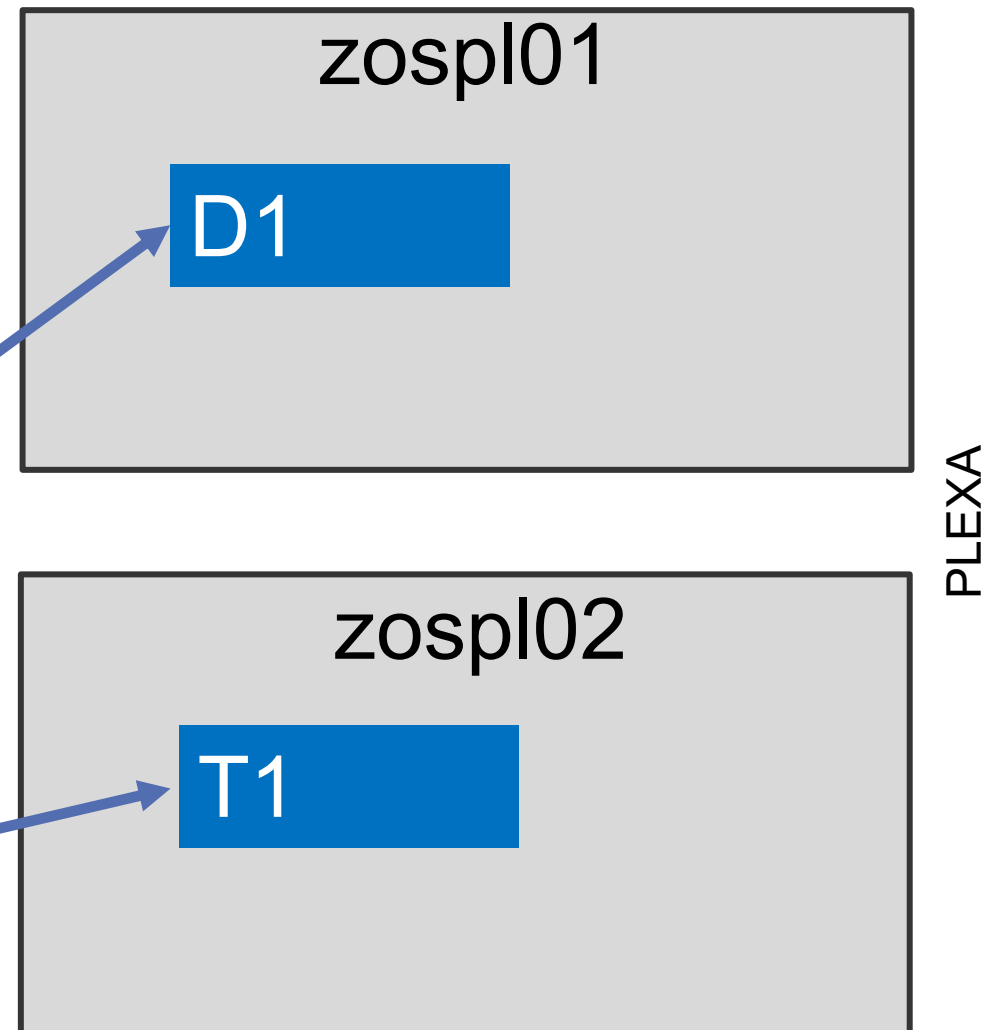
ansible_connection: ssh

ansible_python_interpreter: ...



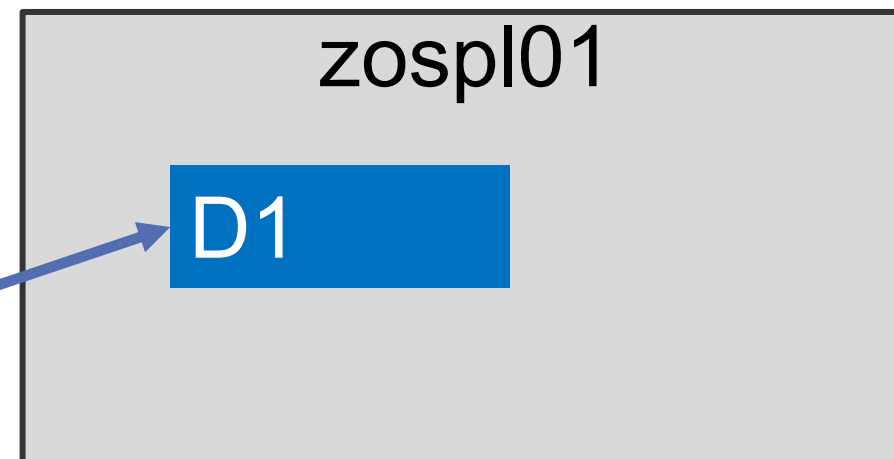
Z/OS SUBSYSTEMS ARE LOGICAL RESOURCES

```
all:
  children:
    zos:
      hosts:
        zoslp01:
          ansible_host: zoslp01.company.com
          sysplex: PLEXA
        zoslp02:
          ansible_host: zoslp02.company.com
          sysplex: PLEXA
      db2_subsystems:
        hosts:
          db2d1:
            zos_host: zoslp01
            db2_ssaid: D1
            db2_location: D1LOC
          db2t1:
            zos_host: zoslp02
            db2_ssaid: T1
            db2_location: T1LOC
```



EXAMPLE: STOPPING DB2 SUBSYSTEMS

```
- name Stop DB2 subsystems
hosts: db2_subsystems
tasks:
- name: Stop DB2
  ibm.ibm_zos_core.zos_operator:
    cmd: "STOP DB2,SSID={{ db2_ssid }}"
    delegate_to: "{{ zos_host }}"
```



STOP DB2 is a z/OS operator command. `zos_operator` simulates a human operator entering the command on the system console.

The variables come from

```
hostvars[inventory_hostname].zos_host
hostvars[inventory_hostname].db2_ssid
```

You can see why you have to understand what's happening on the z/OS system. Ansible is only invoking z/OS functions, it isn't directly manipulating the z/OS instance. If you don't know what those functions are, Ansible will not magically enable you to manage z/OS systems.

THE LAW OF LEAKY ABSTRACTIONS



In 2002, software engineer Joel Spolsky observed that abstractions are *leaky*. He meant that at times, details of the underlying layer leak through the interface and become visible to users of the abstraction.

If your knowledge of a system is limited to the view provided by the abstraction, there will be situations in which you experience difficulty troubleshooting.

His article uses TCP as an example. TCP is designed to be a reliable protocol, but it is built on top of IP, which is an unreliable protocol. He writes that TCP is "what computer scientists like to call an abstraction: a simplification of something much more complicated that is going on under the covers."

The law of leaky abstractions: **"All non-trivial abstractions, to some degree, are leaky."**

Ansible is a non-trivial abstraction that hides much of the complexity of what is happening on a z/OS instance. It does a pretty good job of that, and you will be able to manage z/OS nodes via Ansible most of the time. But there will be situations when the error messages that make it back to the control node are confusing or incomplete, and you must look at the z/OS environment to figure out what's happening.

DOCUMENTATION

For more information, consult the documentation at the following URLs:

Introduction to z/OSMF

<https://www.ibm.com/docs/en/zos/3.1.0?topic=guide-introduction-zosmf>

IBM z/OSMF Ansible collection (ibm_zos_zosmf)

https://ibm.github.io/ibm_zos_zosmf/index.html

Getting Stated With IBM z/OSMF Workflows

<https://www.ibm.com/docs/en/ukofz/3.1.0?topic=how-getting-started-workflows>

Asible sample playbook to start a z/OSMF Workflow

https://ibm.github.io/ibm_zos_zosmf/playbooks/sample_module_workflow.html

DOCUMENTATION

IBM z/OS Provisioning Toolkit

<https://www.ibm.com/products/zos-provisioning-toolkit>

Provisioning a z/OS Software Instance

<https://www.ibm.com/docs/en/zos/3.2.0?topic=services-provisioning-zos-software-instance>

IBM Education materials on cloud provisioning

<https://github.com/IBM/IBM-Z-zOS/blob/main/zOS-Education/zOS-V2.3-Education/IEAV2R3%20zOSMF-IBM%20Cloud%20Provisioning%20and%20Management%20for%20zOS.pdf>

Provision and manage z/OS software instances

https://ibm.github.io/ibm_zos_zosmf/README_cpm.html

IBM Z System Automation

<https://www.ibm.com/products/z-system-automation>

WHAT YOU KNOW AND WHAT YOU CAN DO

- You understand that z/OS has its own facilities for system management and automation, and that Ansible invokes those facilities via ssh to support z/OS managed nodes.
- You understand that non-trivial abstractions may expose (that is, *leak*) details of the underlying layer, and so even when we are using an abstraction we need to be aware of what's happening under the covers. The principle applies when we depend on Ansible as an abstraction over z/OS system management facilities.
- You know where to look for documentation, tutorials, presentations, and other material that may help you come up to speed with Ansible for z/OS.

WHAT'S NEXT

- Time-Sharing Option (TSO)
- Interactive System Productivity Facility (ISPF)